

Model Context Protocol (MCP)

Transformers, uso de agentes, LangChain y RAG

Nicolás Cayturo

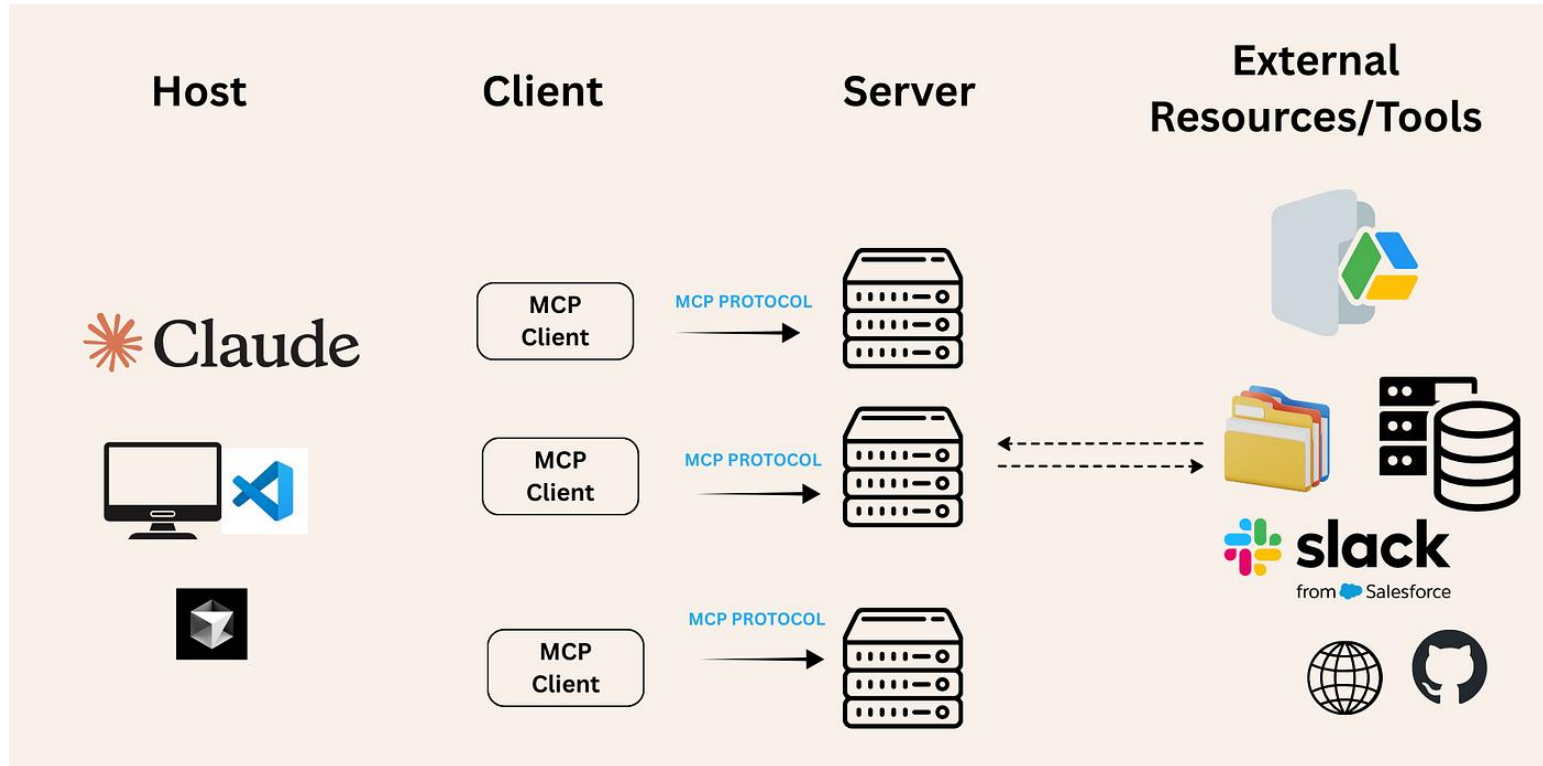
Motivación

Los agentes son “inteligentes”, pero necesitan acceder al mundo exterior para ser realmente útiles. El desafío es que cada aplicación o servicio tiene su propia API y su propia forma de comunicación.

En este contexto, MCP es un protocolo abierto que estandariza cómo las aplicaciones con LLMs se conectan a datos y herramientas.

- Existe una gran cantidad de integraciones ya creadas.
- Provee una manera estandarizada de construir integraciones.
- Es un protocolo abierto. Lo inició Anthropic como un estándar abierto para la comunicación entre modelos de lenguaje (LLMs) y herramientas externas.

Arquitectura General de MCP



Arquitectura General de MCP

Host

Esta es la aplicación de IA con la cual interactuamos como Visual Studio Code, Claude Desktop, etc. Es responsable de gestionar la experiencia general y coordinar la comunicación.

Cliente

Cada cliente MCP es creado por el host para conectarse a un servidor MCP específico. Tiene una relación 1:1 con el servidor (local o remoto). Su trabajo es mantener la conexión y obtener contexto o datos útiles desde el servidor para que el host lo pueda utilizar.

Servidor

Este es el programa o servicio que provee el contexto o datos que la aplicación de IA necesita. Puede ser algo como un servidor que gestione nuestros archivos locales o uno como el servidor MCP de Google.

Comunicación entre componentes MCP

JSON-RPC 2.0

Es un protocolo ligero de llamadas a procedimientos remotos que usan JSON; define cómo estructurar solicitudes y respuestas de forma agnóstica al transporte, siendo la versión 2.0 la más utilizada.

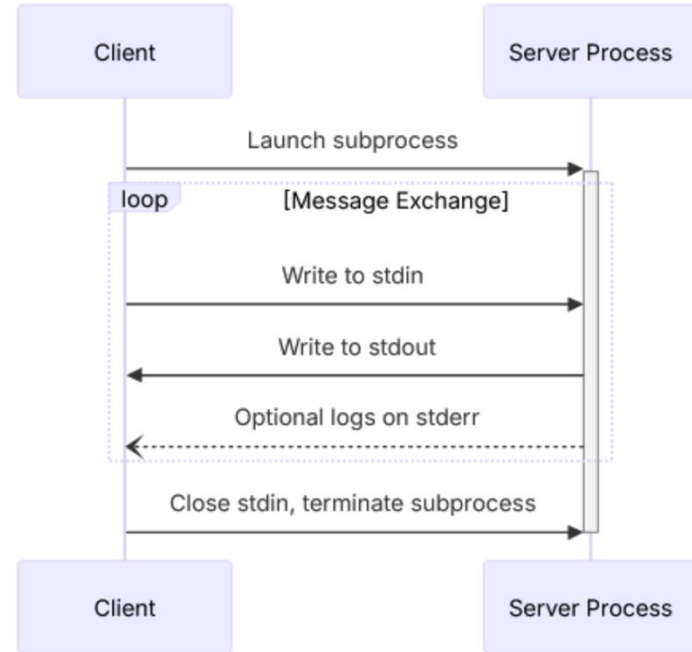
```
{"jsonrpc":"2.0", "id":1, "method":"tools/list", "params":{}}  
{"jsonrpc":"2.0", "id":1, "result":{}}  
{"jsonrpc":"2.0", "method":"notifications/tools/list_changed", "params":{}}
```

Mecanismos de transporte

STDIO (procesos locales)

Es el sistema de flujos estándar de un SO/terminal para la entrada y salida de programas:

- **Componentes:**
 - Stdin: para enviar información
 - Stdout: para recibir respuesta
 - Stderr: para mensajes de diagnóstico
- **Usos comunes:** leer entrada del usuario o de otros procesos y escribir los resultados en pantalla o en un archivo.



Mecanismos de transporte

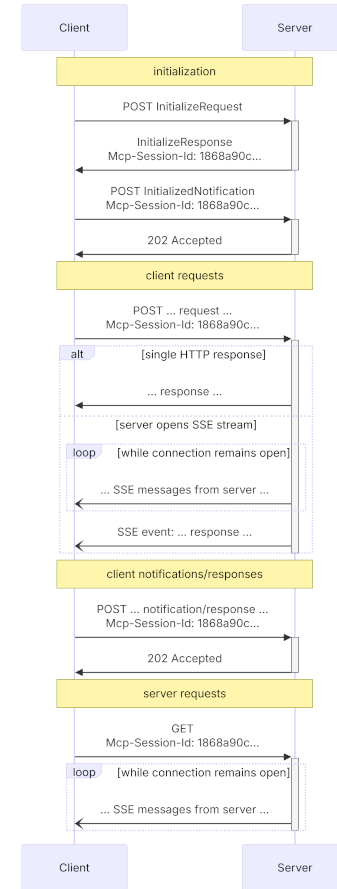
Streamable HTTP (procesos remotos)

El servidor actúa como un proceso independiente que puede manejar distintas conexiones y clientes.

- Usa peticiones POST, GET y HTTP
- Puede usar Server-Sent Event (SSE) para consumir múltiples mensajes del servidor.

SSE es un mecanismo que permite a los clientes web a recibir actualizaciones automáticas desde el servidor. Estas actualizaciones son conocidas como “eventos”, y son enviadas sobre una sola conexión HTTP persistente.

En streamable HTTP la comunicación se hace sin la necesidad de una conexión HTTP persistente. Funciona con HTML puro.



Cliente MCP

Es el componente dentro del host que establece y gestiona sesiones 1:1 con servidores MCP, negociando capacidades, enrutando mensajes JSON-RPC y manteniendo el aislamiento y la seguridad entre servidores.

Funciones principales

- **Negociación:** Define la versión del protocolo y las capacidades compatibles con el servidor.
- **Descubrimiento:** Identifica de forma dinámica las herramientas recursos y plantillas de prompts disponibles en el servidor.
- **Comunicación:** Envía solicitudes estructuradas mediante JSON-RPC a través de los transportes vistos anteriormente.

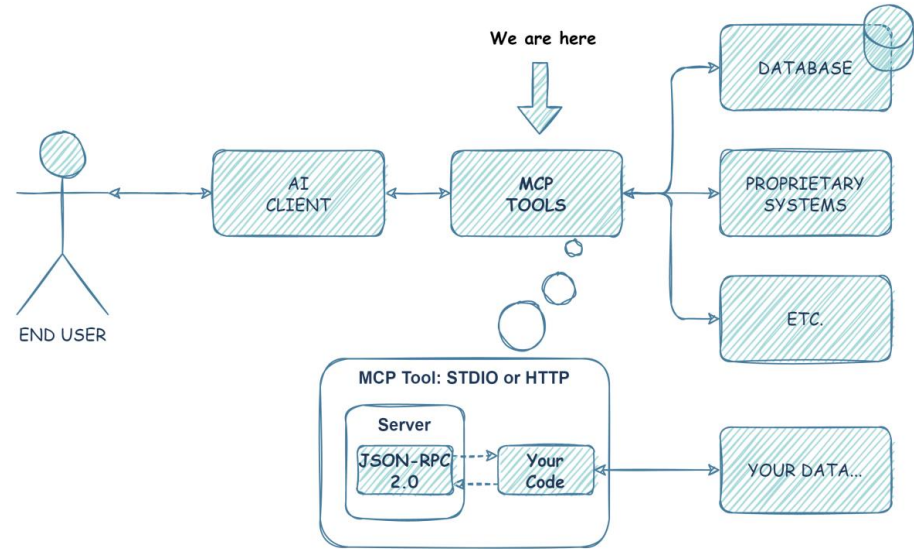
En resumen, orquesta las capacidades de los servidores: descubre y usa prompts, recursos y herramientas; mantiene sesiones seguras, negocia capacidades y actúa como un puente entre el host/LLM y los servidores (locales o remotos).

<https://mcp.so/clients>

Funcionalidades de un MCP (Servidor)

Tools

Las herramientas (tools) son interfaces definidas por esquemas que los LLM pueden invocar. Cada herramienta realiza una única operación con entradas y salidas claramente definidas. Y, lo más importante, la ejecución de una herramienta requiere la aprobación explícita del usuario, garantizando que este mantenga el control sobre las acciones que realiza el modelo.



Funcionalidades de un MCP (Servidor)

Resources

Los recursos proporcionan acceso estructurado a información que la aplicación host puede recuperar y ofrecer a los modelos de IA como contexto.

- Archivos
- APIs
- Bases de datos
- Cualquier otra fuente que una IA necesite como contexto

Las aplicaciones pueden acceder directamente a esta información y decidir como usarla.

Funcionalidades de un MCP (Servidor)

Prompts

Los prompts proporcionan plantillas reutilizables. Permiten a los autores de servidores MCP ofrecer prompts parametrizados para un dominio o mostrar la mejora forma de utilizar el servidor MCP.

- Están controlados por el usuario, por lo que requiere invocación explícita en lugar de activarse automáticamente.
- Pueden ser conscientes del contexto, haciendo referencia a recursos y herramientas disponibles para crear flujos de trabajo integrales.

Un cliente ingresa un ticket en un sistema y el servidor retorna la siguiente respuesta (que luego será procesada por una LLM).

```
@mcp.prompt()
def analizar_ticket(ticket: str):
    return f"""
    Actúa como especialista de soporte técnico.

    Analiza el siguiente ticket:
    {ticket}

    Determina:
    1. El problema principal.
    2. Su prioridad.
    3. Una posible solución.
    4. Los pasos que debería seguir el usuario.
    """
```

Funcionalidades de un MCP (Cliente)

Roots

Permite al cliente definir qué carpetas, archivos o URIs (Uniform Resource Identifier) puede utilizar el servidor MCP. Funciona como un mecanismo de seguridad para limitar el acceso a todos los recursos.

Por ejemplo, un cliente que analiza documentos solo puede acceder a la carpeta **Documentos/***. Si el servidor intenta leer **/Usuarios/Contraseñas/**, el cliente lo bloquea porque está fuera del root.

Funcionalidades de un MCP (Cliente)

Sampling

Permite que el servidor MCP solicite al cliente que utilice un LLM para generar texto, resumir información, razonar o responder preguntas. El servidor no ejecuta el modelo directamente; delega esa tarea al cliente.

Por ejemplo, un servidor obtiene un archivo PDF y solicita al cliente: “**Resume este documento utilizando GPT-5**”. El cliente envía la petición al LLM y devuelve el resumen al servidor.

Funcionalidades de un MCP (Cliente)

Elicitación

Permite que el servidor solicite información adicional al usuario cuando la información disponible no es suficiente para completar una tarea.

Por ejemplo, un usuario le pide que **reserve un vuelo**. El servidor detecta que faltan datos y pide al cliente preguntar **¿cuál es la ciudad de destino y la fecha de viaje?**.

Funcionalidades de un MCP (Cliente)

Discovery

Permite que el cliente descubra dinámicamente las herramientas, recursos y prompts que ofrece un servidor MCP, incluso si estos cambian durante la ejecución.

Por ejemplo, un servidor agrega una nueva herramienta llamada **traducir_documento**. Sin reiniciar el agente, el cliente actualiza automáticamente la lista de herramientas.

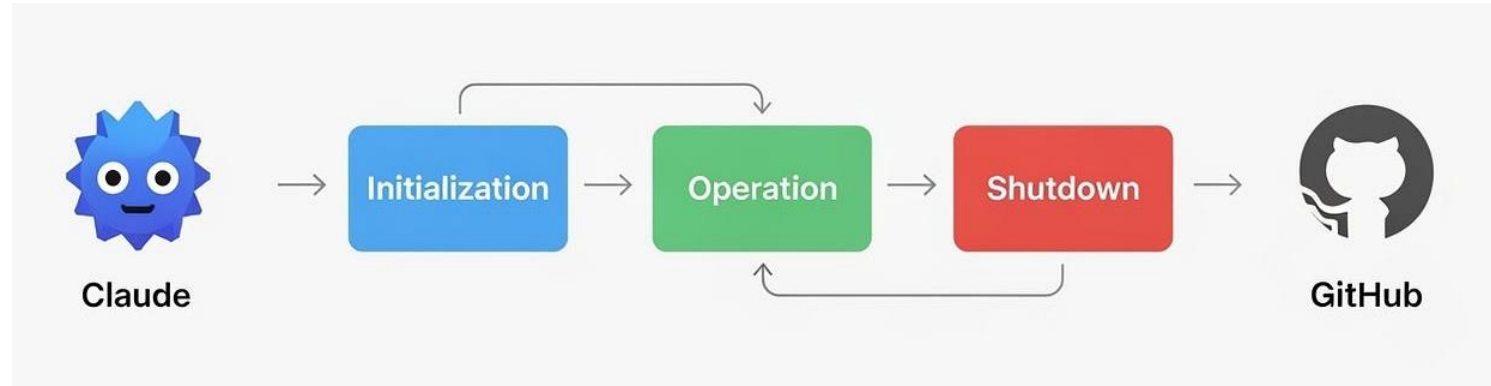
Logging / Progress / Cancel

Permite informar el progreso de una tarea, registrar eventos (logs) y cancelar operaciones largas. Mejora la observabilidad y experiencia del usuario.

Por ejemplo, un agente procesa 100 documentos. Durante el proceso muestra el progreso y permite que el usuario cancele para detener la ejecución.

Ciclo de vida de la conexión C:S MCP

El ciclo de vida de la conexión cliente-servidor MCP se divide en tres fases.



Ciclo de vida de la conexión C:S MCP

Fase de inicialización

En esta fase, el agente de IA (e.g., Claude Desktop) lanza el servidor MCP y acuerdan como van a comunicarse.

- El cliente inicia la conexión.
 - “Hola, uso MCP v1.0”.
- Se intercambia información sobre la versión de MCP.
 - “Yo también uso MCP v1.0”.
- El servidor informa qué capacidades ofrece (tools, resources, prompts).
 - “Tengo estas herramientas: buscar_cliente, generar_reporte”
- El cliente informa qué capacidades soporta (roots, sampling, logging, etc.).
 - “Puedo usar sampling.y roots”.
- Ambos verifican que son compatibles.

Ciclo de vida de la conexión C:S MCP

Fase de operación

En esta fase, el cliente y servidor trabajan intercambiando solicitudes y respuestas.

- El usuario hace una petición.
 - “Busca el cliente Juan Pérez”.
- El cliente llama a una herramienta.
 - Servidor MCP.
- El servidor ejecuta una tarea.
 - Ejecutar herramienta “buscar_cliente()”.
- Puede solicitar sampling.
- Puede pedir más información mediante elicitación.
- Envía el progreso y resultados.

Ciclo de vida de la conexión C:S MCP

Fase de finalización (shutdown)

Esta es la etapa donde la comunicación termina de forma ordenada.

- Se cancelan tareas pendientes.
- Se liberan recursos.
- Se cierran archivos y conexiones.
- Se desconecta el servidor.

Principios de seguridad

MCP pone el control del usuario en el centro y reduce la superficie de riesgo con consentimiento explícito, aislamiento por servidor y buenas prácticas en transporte y autenticación.

- **Consentimiento del usuario.** El usuario aprueba el uso de herramientas y el acceso a recursos.
- **Privacidad.** Solo se comparte información estrictamente necesaria.
- **Control de herramientas.** Las herramientas sensibles requieren autorización.
- **Control de Sampling.** El cliente decide cuándo y con qué LLM se realiza una generación.
- **Roots y Allowlists.** Limitan el acceso únicamente a carpetas y recursos autorizados.
- **Validación de entradas.** Verifican parámetros para evitar datos maliciosos.
- **Logs seguros.** Evitan registrar contraseñas, tokens o información confidencial.
- **CONFIANZA ZERO.** Todo servidor y recurso externo debe considerarse, en principio, potencialmente no confiable.